Eleni Constantinou, George Kakarontzas and Ioannis Stamelos: "Towards Open Source Software System Architecture Recovery Using Design Metrics", PCI 2011, pp. 166-170, IEEE
DOI: http://doi.ieeecomputersociety.org/10.1109/PCI.2011.36

©2011 IEEE. Personal use of this material is permitted. However, permission to reprint/republish this material for advertising or promotional purposes or for creating new collective works for resale or redistribution to servers or lists, or to reuse any copyrighted component of this work in other works must be obtained from the IEEE

Towards Open Source Software System Architecture Recovery Using Design Metrics

Eleni Constantinou
Computer Science Department
Aristotle University of Thessaloniki
Thessaloniki, Greece
econst@csd.auth.gr

George Kakarontzas
Computer Science Department
Aristotle University of Thessaloniki
Thessaloniki, Greece, and
Computer Science and Telecom. Department
T.E.I. of Larissa, Greece
gkakaron@teilar.gr

Ioannis Stamelos
Computer Science Department
Aristotle University of Thessaloniki
Thessaloniki, Greece
stamelos@csd.auth.gr

***Abstract*—Over the past years, software development practices include open source code reuse. Since documentation gives little or no information about the system architecture, a prohibitive amount of effort is required to comprehend the source code and the overall system architecture. In this paper, we investigate how design metrics can reveal architectural information about a software system and more specifically, how architectural layers are correlated to design metrics. Finally, we present an empirical study on two large open source systems written in Java, attempting to identify metrics revealing information about the system architecture.**

***Keywords*-system understanding; program comprehension; object-oriented; reuse; architecture layer; design metrics;**

I. Introduction

In [1] Software Architecture is defined as *'the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both'*. Software architecture is essential for understanding a system with the purposes of reuse, maintenance, quality analysis, etc. However, open source systems often lack such information and developers resort to manual inspection of thousands of lines of code, which makes the understanding process difficult if not impossible. In order to facilitate open source code reuse, the main focus of the EU-funded OPEN-SME project[1], information about the architectural structure must be recovered. For open source projects, where the documentation may be absent or very poor, source code analysis seems a promising approach. In this work we investigate whether source code metrics in object-oriented systems can be used for the recovery of architectural layers [2].

Architecture recovery has been a subject of research for many years, and mainly focused on clustering and pattern-based techniques. Methods based on source code clustering, analyze the relationships among the modules, while pattern-based techniques focus on discovering architectural patterns. For the experimental evaluation of such methods, in some cases, legacy systems are used, where some information about the architecture of the system exists, while the last few years researchers tend to apply their methodologies in open source systems.

Our approach is based on static source code analysis, as the source code is available in every open source system. We investigate if strong correlations between the architectural layers of the system and object-oriented design metrics actually exist. The actual architectural layers of the system are initially unknown. Using the Directed Acyclic Graph (DAG) of the system, as produced by analyzing the static dependencies among classes, we derive what we call D-layers (DAG Layers). D-layers are groups of classes with the characteristic that classes at higher D-layers depend only upon classes in lower D-layers but not the opposite. However classes are classified in architectural layers also in accordance to their actual role in the system (user interface class, domain class etc.) This type of information cannot be recovered by the DAG alone. The main idea of our work is that D-Layers can be combined to form architectural layers, if such layers exist in the system under analysis. We make the conjecture that classes in different architectural layers exhibit statistically significant differences in their design metrics' values. Since D-layers provide an initial under-approximation of the actual layers, we combine consecutive D-layers together to form *tentative* architectural layers. We then test our conjecture by extracting the popular Chidamber and Kemerer [3] metrics from the source code and study if there are correlations between their values and the values of the formed architectural layers. Finally, we apply machine learning techniques, more specifically the JRip classification algorithm, to generate classification rules relating metric values to architecture layers.

The rest of the paper is organized as follows. In Section II we describe related work on architecture recovery. Next, in Section III the proposed methodology is introduced. In Section IV we provide and explain our experimental results. Finally, in Section V, we discuss future research directions

[1] http://opensme.eu/

and conclude.

II. Related Work

Architecture recovery techniques can be distinguished in two categories: clustering-based and pattern-based. Architecture recovery techniques based on clustering of the classes vary. Anquetil et al. [4] perform class clustering according to name similarity. Their experimental results demonstrate that static analysis based on name similarity used as clustering criterion can provide good results for architecture recovery. Zhang et al. [5] define Weighted Directed Class Graph (WDCG) to represent object-oriented software. In particular, they propose a new hybrid clustering algorithm and partition clustering for recovering high-level software architecture from WDCG. They introduce four metrics to validate the effect of their approach.

In an earlier study of Mancoridis et al. [6], algorithms based on hill-climbing and genetic algorithms are implemented to automatically recover the modular structure of a software system from its source code. In a more recent work [7] they examine the results produced by the Bunch clustering system, which uses search techniques to perform clustering. In particular, Bunch decomposes the system by partitioning the graph of entities and relations in the source code, using a fitness function to evaluate the quality of graph partitions and search algorithms to find a satisfactory solution.

Bojic and Velasevic [8] focus on program understanding and reuse engineering of legacy object-oriented systems. Their proposed method is based on dynamic analysis of the system for selected test cases that cover relevant use cases. They use formal concept analysis to decompose the logical hierarchy of subsystems, aiming to group parts of the system that implement similar functionality. However, they do not provide evaluation studies to confirm the validity of their approach.

Sartipi [9] uses data mining techniques to derive relations among entities of the system based on their static structural properties, according to predefined architectural patterns. Vasconcelos et al. [10] investigate ArchMine, an architecture recovery approach based on dynamic analysis and data mining, and integrate it with ArqCheck and ArchToDSSA, an architectural evaluation approach based on inspection and domain variability approach respectively. They perform a series of studies to evaluate the integration between ArchMine and ArqCheck results.

III. Methodology

Modern software systems that follow the layered architectural style usually consist of four layers: *User Interface* (4), *Controllers* (3), *Business Logic* (2) and *Infrastructure* (1). We expect that the classes participating in each layer will have different functional behavior and dependencies with classes in other layers. User Interface layer contains classes that interact with system users and present information. Controller layer classes interact with classes in User Interface to provide information and with classes in Business Logic layer, where domain-specific classes are located. At the bottom of the hierarchy, Infrastructure layer classes are mainly responsible for data storage and other general-purpose services (e.g. logging). We aim to discover classes' architecture layer, using design metrics as indicators of these layers. Our methodology can be described in the following steps:

1) Form the DAG of the system using static analysis, thus deriving D-layers.
2) Arbitrarily combine consecutive D-layers together to form four groups of *tentative* architectural layers
3) Calculate the Chidamber and Kemerer metrics for each of the classes of the system.
4) Calculate the correlation of the indicative metrics of architectural layers to the *current architectural partition*
5) Iteratively keep moving boundary D-layers from their current architectural layers to the adjacent layer until the correlation of the indicative metrics stops improving.

We expect that this approach, when fully developed, will recommend architectural layers of systems following the architectural style automatically, since all steps are automatic (i.e. DAG formation, metrics calculation, correlation calculation, and D-layers moves). However this approach can only work if a subset of the Chidamber and Kemerer metrics, are indeed indicators of architectural layers. In this work we try to answer this question.

Next, we introduce our analysis methodology for this work and the derived results.

We use the Classycle Analyzer [2] tool to discover class dependencies and D-layers. Classycle tool discovers strong dependencies between classes and packages, and creates a Strongly Connected Components (SCC) graph according to Tarjan's algorithm [11]. Next, according to SCCs calls, the graph is condensated to an acyclic digraph of SCCs, from which the D-layers are extracted[3], as shown in Fig. 1.

We include metrics that have received considerable attention in research practices, proposed by Chidamber and Kemerer [3]. All metrics measure class-level properties and are calculated by the Chidamber and Kemerer Java Metrics (ckjm) software package [12]. The aforementioned metrics are briefly described below.

WMC Weighted methods per class. The ckjm tool counts WMC as the total number of declared methods in each class, since it assigns a complexity value of 1 to each method.

[2] http://classycle.sourceforge.net

[3] Actually in the Classycle documentation D-layers are called layers, but as the documentation of the tool points out, these are not layers in the architectural sense. To avoid confusion in this work we took the liberty of renaming Classycle's layers to D-layers (as in DAG layers)

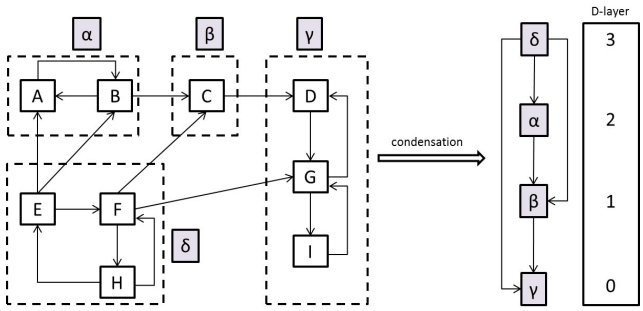

Figure 1. Classycle Acyclic Digraph Creation

DIT Depth of Inheritance Tree. DIT metric provides for each class a measure of the inheritance levels from the object hierarchy top.

NOC Number Of Children. NOC measures the number of immediate descendants of the class.

CBO Coupling Betweeng Object classes. CBO represents the number of classes coupled to a given class.

RFC Response For a Class. Ckjm measures RFC by inspecting method calls within the class's method bodies.

LCOM Lack of Cohesion in Methods. LCOM is calculated by subtracting from the number of method pairs that do not share a field access the number of method pairs that do.

Ca Afferent Coupling. Ca measures how many other classes use the specific class.

NPM Number of Public Methods. NPM counts all the methods in a class that are declared public to measure the size of an API provided by a package.

Our goal is to empirically analyze the design metrics in order to evaluate whether or not these metrics can predict accurately the architecture layer of a class. Initially, we investigate if correlations between the architectural layer and design metrics exist using Spearman correlation coefficient.

Next, the D-layer values extracted by the data are binned into four categories, each corresponding to a *tentative* architecture layer. Since we aim to classify each class to one of these layers, metrics values are also binned into categories. The target variable D-layer is binned into four categories according to the range of its values producing a categorical variable layerBin. Metric variables correlated to layer are binned according to the new variable layerBin, while the remaining metrics are ignored.

Since the analysis reveals strong correlations between the layer and several metrics, also confirmed in the binned values, we extract classification rules implemented in Open Source Machine Learning Software Weka [13]. The algorithm used is JRip and the extracted rules are aimed to predict the architecture layer a class is classified according to the binned values of the metrics.

To validate our results, we use the following measures for each architecture layer:

Precision: the number of correctly classified instances, divided by the total number of instances classified in this layer.

Recall: the number of correctly classified instances, divided by the total number of instances of this layer.

Table I
DESCRIPTIVE STATISTICS FOR OPENPROJ

	Min	Max	Mean	Std. Deviation
D-layer	0	18	8.6	6.897
WMC	0	374	9.10	21.916
DIT	1	9	2.06	1.677
NOC	0	48	.25	1.972
CBO	0	162	5.12	9.286
RFC	0	715	22.70	47.716
LCOM	0	67883	219.792	2557.792
Ca	0	179	4.67	11.781
NPM	0	355	7.70	20.193

Table II
DESCRIPTIVE STATISTICS FOR RHINO

	Min	Max	Mean	Std. Deviation
D-layer	0	18	6.25	4.558
WMC	0	193	10.90	17.597
DIT	1	6	1.93	1.385
NOC	0	21	.31	1.311
CBO	0	55	4.24	6.027
RFC	0	394	27.45	42.878
LCOM	0	18392	160.72	959.731
Ca	0	123	4.20	9.808
NPM	0	156	6.02	11.684

IV. RESULTS

Next, we provide a description of the data used for the validation of the proposed method. We apply our method to two large open source systems written in Java, OpenProj and Rhino. OpenProj [4] is a popular free, open source project management solution. We used version 1.4 which consists of 1364 classes. Rhino [5] is an open-source implementation of JavaScript written entirely in Java, usually embedded into Java applications to provide scripting to end users. We used version 1.7 which consists of 523 classes.

We extract classes' D-layers and design metrics for the two projects, using Classycle and ckjm respectively. Some descriptive statistics are shown for OpenProj and Rhino in Table I and Table II respectively.

Next, we examine the correlations among the variables performing Spearman correlation analysis. The results are illustrated for datasets OpenProj and Rhino in Table III and Table IV, respectively. Metrics DIT, CBO, LCOM, Ca and NPM are related to the D-layer variable in both systems. WMC variable is correlated to the D-layer in OpenProj, how-

[4]http://openproj.org
[5]http://www.mozilla.org/rhino

Table III
CORRELATIONS FOR OPENPROJ

	layer	WMC	DIT	NOC	CBO	RFC	LCOM	Ca	NPM
D-layer	1	-.063*	.338*	-.040	.443*	.175*	-.152*	-.196*	-.133*
WMC		1	-.012	.223*	.430*	.815*	.808*	.448*	.941*
DIT			1	.046	.252*	.150*	-.052	-.069*	-.012
NOC				1	.080*	.158*	.175*	.255*	.215*
CBO					1	.679*	.291*	.151*	.318*
RFC						1	.604*	.313*	.721*
LCOM							1	.431*	.779*
Ca								1	.461*
NPM									1

Table IV
CORRELATIONS FOR RHINO

	layer	WMC	DIT	NOC	CBO	RFC	LCOM	Ca	NPM
D-layer	1	-.043	.213*	-.008	.410*	.184*	-.122*	-.160*	-.125*
WMC		1	.110*	.254*	.563*	.916*	.755*	.412*	.750*
DIT			1	.054	.215*	.215*	.087*	-.057	.115*
NOC				1	.194*	.234*	.225*	.348*	.228*
CBO					1	.6722*	.463*	.202*	.322*
RFC						1	.672*	.313*	.654*
LCOM							1	.427*	.570*
Ca								1	.385*
NPM									1

ever the strength of the correlation is insignificant compared to the strength of the other correlations.

To further investigate the correlations revealed in the previous results, a new set of experiments follows. D-layer and all variables correlated to D-layer are binned into categorical variables. The dataset D-layer values range from 0 to 18 and the binned values are mapped to four categories of layerBin, one per tentative architecture layer. Next, all the correlated variables are binned according to their variance in respect of the layerBin and the respective variables are created. Table V and Table VI summarize the binned values for OpenProj and Rhino, respectively. The correlations between the binned variables and layerBin (not shown here due to lack of space) remain and in fact are reinforced after the transformation.

Classification rules generation was considered to predict the architecture layer of a class. We imported the binned dataset in Weka and performed JRip classification algorithm.

The classification rules generated for each project are the following:

OpenProj Rules

Rule 1. IF *CaBin = 1* and *CBOBin = 3* and *DITBin = 1* THEN *layerBin=2*

Rule 2. IF *WMCBin = 2* and *LCOMBin* = 1 THEN *layerBin=4*

Rule 3. IF *CBOBin = 3* and *DITBin = 3* THEN *layerBin=4*

Rule 4. IF *DITBin = 2* and *CBOBin = 3* THEN *layerBin=4*

Rule 5. IF *CaBin = 2* and *NPMBin = 2* and *WMCBin = 3* and *LCOMBin = 1* THEN *layerBin=4*

Rule 6. ELSE *layerBin=1*

Rhino Rules

Rule 1. IF *CBOBin = 1* and *DITBin = 1* THEN *layerBin=1*

Rule 2. ELSE *layerBin=2*

Table V
OPENPROJ BINNED VALUES

	1	2	3	4
layer	0-4	5-9	10-14	15-18
WMC	<2	2-4	>=4	
DIT	<4	4-7	>=7	
CBO	<1	1-2	>=2	
RFC	<4	>=4		
LCOM	<1	>=1		
Ca	<1	1-2	>=2	
NPM	<1	1-4	>=4	

Table VI
RHINO BINNED VALUES

	1	2	3	4
layer	0-4	5-9	10-14	15-18
WMC	<12	>=12		
DIT	<2	>=2		
CBO	<2	>=2		
RFC	<20	>=20		

Interpreting the generated rules, we verify that they reveal to some extent information about classes behavior attributes in respect to their architecture layer. For example, Rule 1 demonstrates behavior of low architecture classes. To validate the results of the classification rules we performed 10-fold cross validation. The classification method achieves 73.39% accuracy for OpenProj instances, 1001 out of 1364 classes, and for Rhino instances 56.21%, 294 out of 523 classes.

However, the rules generated are not efficient enough to represent all types of correlations discovered. An indicative example is the one of Rule 1 and Rule 5 of OpenProj, where Rule 1 verifies the initial negative correlation between Ca and D-layer but Rule 5 does not retain it. On the contrary, binned variables correlations maintain and improve the correlations. In our ongoing work therefore, we plan (as explained in Sec. III) to give priority in a correlation-driven approach, whereas at the same time we plan on examining ways to improve the machine learning approach and derive more accurate rules.

In terms of precision and recall results are depicted in Table VII and Table VIII for OpenProj and Rhino, respectively. CK metrics reveal information about system architecture. The results are more fitting to what was expected for OpenProj. Since OpenProj and Rhino are two completely different types of systems this is expected. OpenProj is a typical desktop application with higher probability of following the

Table VII
ACCURACY MEASURES FOR OPENPROJ

	Precision	Recall	ROC Area
layerBin=1	0.708	0.777	0.757
layerBin=2	0.729	0.319	0.7
layerBin=3	0	0	0.663
layerBin=4	0.722	0.734	0.796

Table VIII
ACCURACY MEASURES FOR RHINO

	Precision	Recall	ROC Area
layerBin=1	0.54	0.639	0.698
layerBin=2	0.584	0.741	0.598
layerBin=3	0.143	0.012	0.579
layerBin=4	0	0	0.557

layered architectural style, whereas Rhino is a JavaScript impementation which is basically intended to be embedded in Java applications. The inability of the metrics to predict Layer-3 classes in OpenProj can be attributed intuitively to the fact that this is the initial architectural decomposition of D-layers to architectural layers, which as we said already is *tentative*.

V. CONCLUSION AND FUTURE RESEARCH DIRECTIONS

In this paper, we investigated if object-oriented design metrics can reveal information about the architecture layer. We performed static analysis to recover the tentative architectural layers of the system, using graph clustering and graph transformation techniques. Next, we used CK metrics extraction for system classes and classification rules were generated to predict the architecture layer of each class, as a final step of our method.

We evaluated our analysis in two open source systems and the experimental results are encouraging. We discovered that design metrics, and more specifically CBO, DIT, WMC, RFC, LCOM, Ca and NPM, contribute to architectural recovery. Work is underway to test the full methodology described in Sec. III, in many systems with different types (typical desktop applications, headless (no-UI) applications, libraries etc.) and verify our methodological assumptions.

Concluding, the insight in software architecture gained by this study has proven beneficial for architecture layer recovery. Other aspects that we plan to research include the following: (a) In our analysis we consider that each metric contributes equally to architecture layer prediction. Weight attachment to metric contribution could lead to better results, (b) We also plan to explore additional factors, e.g. conceptual coupling, capable of improving our analysis and achieving better results in terms of precision and recall.

ACKNOWLEDGMENT

This work is partially funded by the European Commission in the context of the OPEN-SME "Open-Source Software Reuse Services for SMEs" project, under the grant agreement no. FP7-SME-2008-2/243768.

REFERENCES

[1] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord and J. Stafford, *Documenting Software Architectures: Views and Beyond*, 2nd ed. Addison-Wesley, 2010

[2] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad and M. Stal, *Pattern-Oriented Software Architecture, Volume 1, A System of Patterns*, Wiley, 1996

[3] S. R. Chidamber and C. F. Kemerer, *A Metrics Suite for Object-Oriented Design*, IEEE Transactions on Software Engineering, Vol. 20, No. 5, May 1999, pp 476-493.

[4] N. Anquetil and T. C. Lethbridge, *Recovering Software Architecture from the Names of Source Files*, Journal of Software Maintenance: Research and Practice, Vol. 11, No. 3, May 1999, pp 201-221.

[5] Q. Zhang, D. Qiu, Q. Tian and L. Sun *Object-Oriented Software Architecture Recovery Using a New Hybrid Clustering Algorithm*, Seventh International Conference on Fuzzy Systems and Knowledge Discovery, Vol. 6, 2010, pp 2546-2550.

[6] S. Mancoridis, B. S. Mitchell, C. Rorres, Y. Chen and E. Gansner, *Using Automatic Clustering to Produce High-Level System Organizations of Source Code*, International Workshop on Program Comprehension, IEEE Computer Society Press, 1998, pp 45-52.

[7] B. S. Mitchell and S. Mancoridis, *On the Automatic Modularization of Software Systems Using the Bunch Tool*, IEEE Transactions on Software Engineering, Vol. 32, No. 3, 2006, pp 193-208.

[8] D. Bojic and D. Velasevic, *A Use-Case Driven Method of Architecture Recovery for Program Understanding and Reuse Reengineering*, Conference on Software Maintenance and Reengineering, 2000, pp 23-32.

[9] K. Sartipi, *Software Architecture Recovery based on Pattern Matching*, Ph.D. Thesis, School of Computer Science, University of Waterloo, 2003.

[10] A. Vasconcelos and C. Werner, *Evaluating Reuse and Program Understanding in ArchMine Architecture Recovery Approach*, Information Sciences, Vol. 181, No. 13, 2011, pp 2761-2786.

[11] R. Tarjan, *Depth-first Search and Linear Graph Algorithms*, Siam Journal on Computing, Vol. 1, No. 2, 1972, pp 146-160.

[12] D. Spinellis, *Tool Writing: A Forgotten Art?*, IEEE Software, Vol. 22, No. 4, 2005, pp 9-11.

[13] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I.H. Witten, *The WEKA data mining software: An update*, SIGKDD Explorations, Vol. 11, No. 1, 2009, pp 10-18.